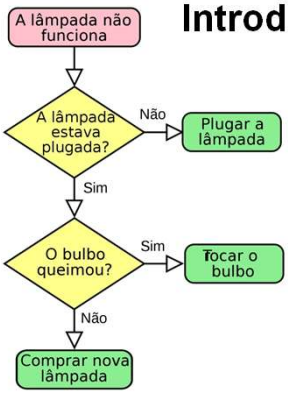
# Universidade Estadual do Piauí – UESPI
# Campus de Parnaíba
# Tecnologia em Sistemas de Computação




## Introdução a Computação

## Aula 15 – Algoritmos e Resolução de Problemas

**Prof. Francisco Rocha**





## LP de Baixo Nível e Pseudocódigo

- **Objetivos**

– descrever o processo de resolução de problemas computacionais e relacioná-lo à lista "Como Resolver" de Polya.
– distinguir entre um tipo simples e um tipo composto.
– descrever três mecanismos compostos de estruturação de dados.
– reconhecer um problema recursivo e escrever um algoritmo recursivo para resolvê-lo.
– distinguir entre um array não ordenado e um array ordenado.
– distinguir entre uma ordenação por seleção e uma ordenação por inserção.
– descrever o algoritmo Quicksort.
– aplicar manualmente a ordenação por seleção, a ordenação por bolha, a ordenação por inserção e o Quicksort a um array de itens.
– aplicar o algoritmo de busca binária.
– demonstrar sua compreensão dos algoritmos deste capítulo simulando-os manualmente com uma sequência de itens.

## Algoritmos e Resolução de Problemas

- **Resolução de Problemas**

– Na área de computação, a resolução de problemas trata do ato de encontrar uma solução para uma questão complexa, angustiante, incômoda ou pendente.

– Como você define a resolução de problemas?

– *How to Solve It: a new aspect of mathematical method* ("Como Resolver Problemas: um novo aspecto do método matemático“) de George Polya.

– Trata-se de uma "lista de como resolver problemas", elaborada no contexto de questões matemáticas.
  - No entanto, essa lista é bastante abrangente.

– **Podemos utilizá-la para resolver problemas relacionados à computação!**

3



## Algoritmos e Resolução de Problemas

- **Resolução de Problemas**

– Como você resolve problemas?

  - Compreenda o problema
  - Elabore um plano
  - Execute o plano
  - Analise o resultado

4

## Algoritmos e Resolução de Problemas

- **Resolução de Problemas: estratégias**

– Faça perguntas!

- O que conheço sobre o problema?
- Quais informações preciso processar para encontrar a solução?
- Como é a solução?
- Que tipos de casos especiais existem?
- Como saberei que encontrei a solução?

5



## Algoritmos e Resolução de Problemas

- **Resolução de Problemas: estratégias**

– Faça perguntas! Nunca reinvente a roda!

- Problemas semelhantes surgem repetidamente, sob diferentes formas.
- Um bom programador reconhece uma tarefa – ou subtarefa – que já foi resolvida anteriormente e aplica a solução existente.

– **Consegue pensar em dois problemas semelhantes?**

6

## Algoritmos e Resolução de Problemas

- **Resolução de Problemas: estratégias**

– **Dividir para conquistar!**

- Divida um problema grande em unidades menores e resolva cada um desses problemas menores.
  - – Aplica o conceito de abstração.
  - – A estratégia de "dividir para conquistar" pode ser aplicada repetidamente até que cada subtarefa se torne gerenciável.





## Algoritmos e Resolução de Problemas

- **Resolução de Problemas Computacionais**

– **Fase de análise e especificação**
- Análise
- Especificação

– **Fase de desenvolvimento de algoritmos**
- Desenvolver algoritmo
- Testar algoritmo

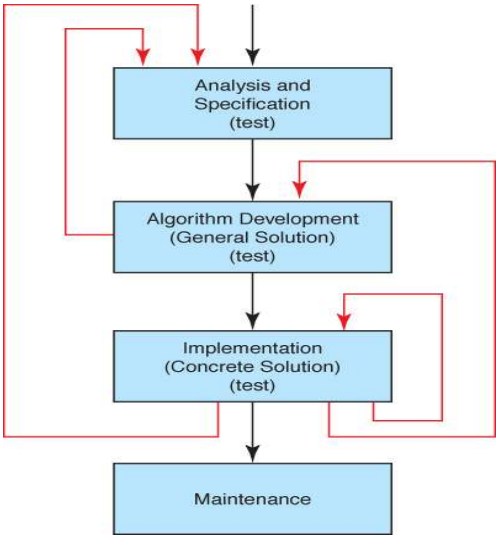
– **Fase de implementação**
- Codificar algoritmo
- Testar algoritmo

– **Fase de manutenção**
- Uso
- Manutenção

- **Você consegue identificar um tema recorrente?**

## Algoritmos e Resolução de Problemas

- **Interação entre Fases**




- Devemos adicionar mais uma seta?
- O que acontece se o problema for revisado?

9



## Algoritmos e Resolução de Problemas

- **Algoritmo**

– Algoritmo é um conjunto de instruções inequívocas para resolver um problema ou subproblema em um tempo finito, utilizando uma quantidade finita de dados.

– Algoritmo é composto de

- **Passo abstrato**
  – Um passo algorítmico que contém detalhes não especificados, i.e., um passo geral com passos internos transparentes.
- **Passo concreto**
  – Um passo algorítmico no qual todos os detalhes estão especificados.

10

## Algoritmos e Resolução de Problemas

- **Desenvolvimento de um Algoritmo**

– Duas metodologias utilizadas para desenvolver soluções computacionais para um problema

- O **projeto top-down** (de cima para baixo) concentra-se nas tarefas a serem realizadas.
- O **projeto orientado a objetos** concentra-se nos dados envolvidos na solução.





## Algoritmos e Resolução de Problemas

- **Resumo da Metodologia**

– Analisar o problema
- Compreender o problema!!
- Desenvolver um plano de ação

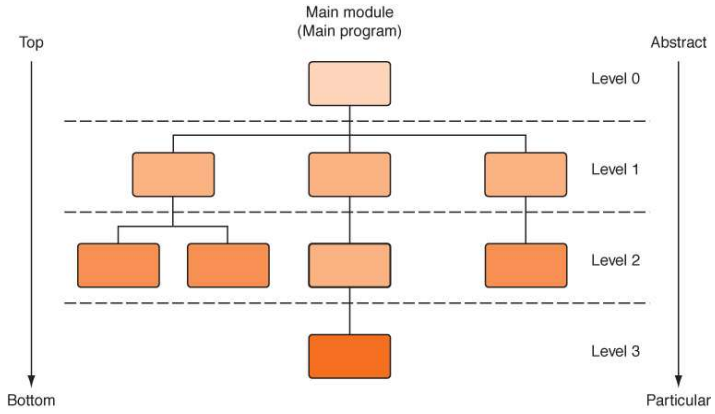
– Listar as tarefas principais (que se tornam o módulo principal)
- Reformular o problema como uma lista de tarefas (módulos)
- Atribuir um nome a cada tarefa

– Elaborar os demais módulos
- Desdobrar cada módulo abstrato em uma lista de tarefas
- Atribuir um nome a cada tarefa

– Reordenar e revisar conforme necessário
- O processo termina quando todas as etapas (módulos) estiverem definidas de forma concreta

## Algoritmos e Resolução de Problemas

- **Projeto Top-Down**




– O processo continua por quantos níveis forem necessários para tornar cada etapa concreta
– O nome de um (sub)problema em um determinado nível torna-se um módulo no nível imediatamente inferior.

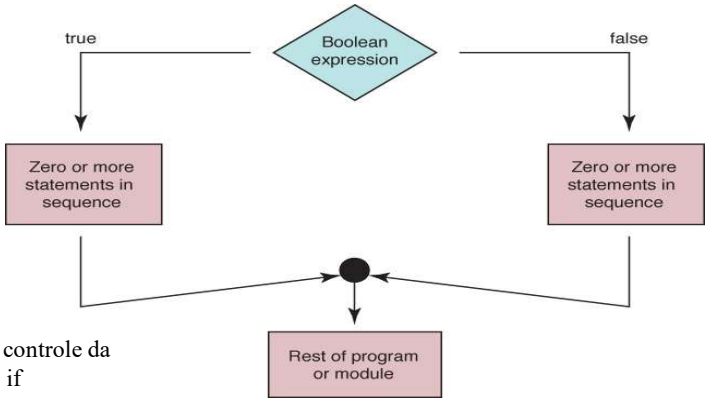




## Algoritmos e Resolução de Problemas

- **Estrutura de Controle**

– Uma **estrutura de controle** é uma instrução que determina a ordem em que outras instruções de um programa são executadas.




Fluxo de controle da instrução if

## Algoritmos e Resolução de Problemas

- **Algoritmo com Seleção**

– **Problema**: Determinar a vestimenta adequada para uma determinada temperatura.

```
Write "Entre com a temperature: "
Read temperature
Determine Dress
```

– Quais afirmações são concretas?

– Quais afirmações são abstratas?





## Algoritmos e Resolução de Problemas

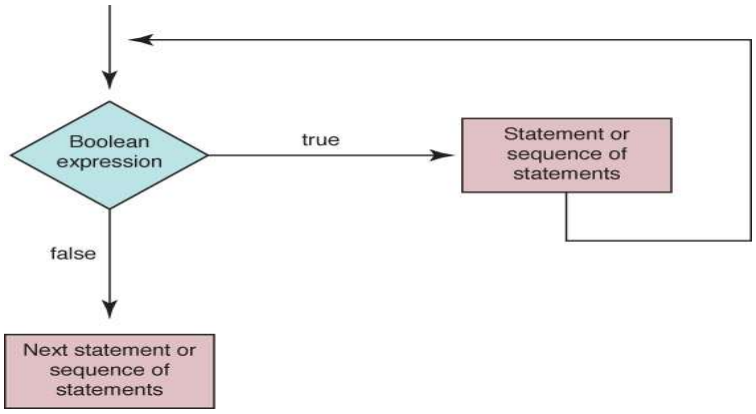
- **Algoritmo com Seleção**

– **Determine Dress**

```
IF (temperature > 40)
   Write "Clima quente: use shorts"
ELSE IF (temperature > 30)
   Write "Clima ideal: mangas curtas são suficientes"
ELSE IF (temperature > 20)
   Write "Pouco frio: use uma jaqueta leve"
ELSE IF (temperature > 5)
   Write "Clima frio: use um casaco pesado"
ELSE
   Write "Fique em casa"
```

## Algoritmos e Resolução de Problemas

- **Comandos de repetição**




Fluxo de controle do comando while





## Algoritmos e Resolução de Problemas

- **Comandos de repetição**

– Um loop controlado por contagem:

```
sum = 0
count = 1
While (count <= limit)
   Read number
   sum = sum + number
   count += 1
Write "Somatório é " + sum
```

– Por que é chamado de laço controlado por contagem?

## Algoritmos e Resolução de Problemas

- **Comandos de repetição**

– Um loop controlado por evento:

```
sum = 0
allPositive = true
WHILE (allPositive)
   Read number
   IF (number > 0)
      sum = sum + number
   ELSE
      allPositive = false
Write "Somatório é " + sum
```

– Por que é chamado de laço controlado por evento?
– Qual é o evento?

19



## Algoritmos e Resolução de Problemas

- **Tipos de Dados Compostos**

– **Record** (ou Registro)

- Uma coleção nomeada de itens heterogêneos, na qual itens individuais são acessados por meio de um nome. Por exemplo, poderíamos agrupar itens como nome, idade e salário em um registro chamado "Funcionário".

– **Array** (ou Vetor)

- Uma coleção nomeada de itens homogêneos, na qual um item individual é acessado por meio de sua posição (índice) dentro da coleção.

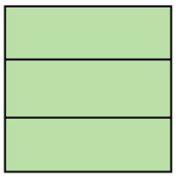
20

## Algoritmos e Resolução de Problemas

- **Tipos de Dados Compostos: record**

– Seguindo o algoritmo, armazena valores nos campos do registro:

**Employee**




```
// Declara uma variável do tipo Employee
Employee employee
employee.name = "Anne Jones"
employee.age = 32
employee.salary = 7500.00
```

21



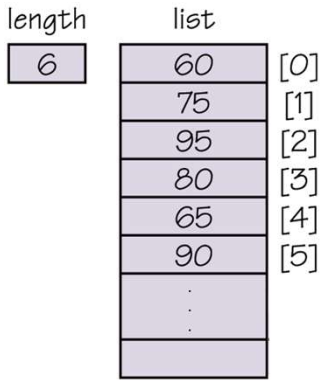
## Algoritmos e Resolução de Problemas

- **Tipos de Dados Compostos: array**

– À medida que os dados são lidos para dentro de um array, um contador é atualizado, de modo que saibamos sempre quantos itens de dados foram armazenados.

– Se o array for chamado de **list**, o intervalo com o qual trabalhamos vai de **list[0]** até **list[length-1]**.

– Os itens de data[0] ... data[length-1] é de interesse.




22

## Algoritmos e Resolução de Problemas

- **Tipos de Dados Compostos: array**

– Algoritmo que preencha um array de números.

```
integer data[20]
Write "Quantos Valores?"
Read length
index = 0
WHILE (index < length)
   Read data[index]
   index = index + 1
```





## Referencias Bibliográficas


– Nell Dale; John A. Lewis. Computer Science Illuminated. 6ed. Jones and Bartlett, 2016.

– Behrouz Forouzan e Firouz Mosharraf. Fundamentos da Ciência da Computação. 2ed. Cengage Learning, 2012.

– J. Gleen Brookshear. Ciência da Computação: uma visão abrangente. 11ed. Bookman, 2013.

– Marcelo Marcula; Pio Armando Benini Filho. Informática: conceitos e aplicações. 4ed. Érica, 2013.

– Ricardo Daniel Fedeli; Enrico Giulio Franco Polloni; Fernando Eduardo Peres. Introdução à ciência da computação. 2ed. Cengage Learning, 2010.


DÚVIDAS


QUEM NÃO ENTENDEU, LEVANTE A MÃO
O QUE VOCÊ NÃO ENTENDEU, MANOLITO?
NADA, DESDE MARÇO ATÉ AGORA



25